

ARM PrimeCell

SDRAM Controller (PL170)

Integration Manual

ARM PrimeCell SDRAM Controller (PL170) Integration Manual

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell SDRAM Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	Bus Interconnection	5
4.4	Endianness	5
4.5	Customisation	6
4.5.1	SDRAM Timing Parameters	6
4.5.2	Other parameters	6
4.6	Synthesis	7
4.6.1	Setup Scripts	7
4.6.2	Command Script	7
4.6.3	Constraint Scripts	8
4.7	Integration with AMBA Micropack components	8
4.8	Clocking	8
4.9	Clock Tree synthesis	9
4.10	Resets	9
4.11	Test Methodologies	9
4.11.1	Scan Insertion and ATPG	9
APPENDIX A	ADDRESS MAPPING	10
A.1	Address Mapping turned ON	10
A.1.1	Address mapping	10
A.2	Address Mapping turned OFF	12

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	6 December, 1999	First release
B	25 January, 2000	Updated for 256Mbit device support
C	10 April, 2000	Updated for BigEndian support

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0159	ARM PrimeCell SDRAM Controller (PL170) Technical Reference Manual
3	PL170 DDES 0000	ARM PrimeCell SDRAM Controller (PL170) Design Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
AHB	Advanced High-performance Bus
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
STA	Static Timing Analysis
SDRAM	Synchronous Dynamic Random Access Memory

2 SCOPE

This document describes how the ARM PrimeCell SDRAM Controller may be integrated into a typical industry standard ASIC design flow.

3 INTRODUCTION

The SDRAM Controller is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on the AMBA AHB, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell SDRAM Controller block, refer to the ARM PrimeCell SDRAM Controller (PL170) Technical Reference Manual [Ref.2].

The ARM PrimeCell SDRAM Controller (PL170) Design Manual [Ref. 3] contains information about the implementation and design of the SDRAM Controller that may be needed for optional customisation of this block.

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell SDRAM Controller has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell SDRAM Controller block.

An example ASIC design flow is described below:

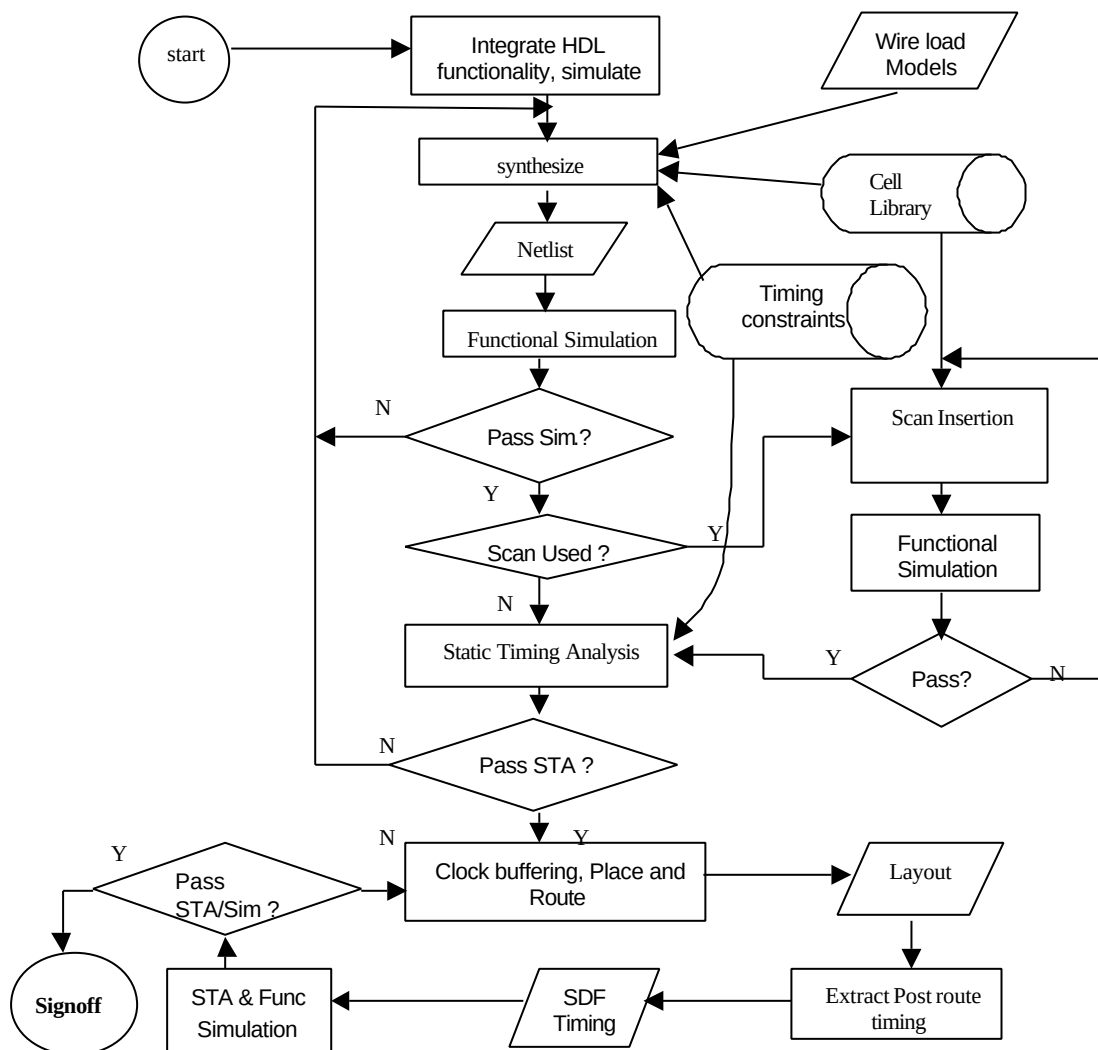


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell SDRAM Controller block. For further information refer to ARM PrimeCell SDRAM Controller (PL170) Technical Reference Manual [Ref.2].

Signal Name	Type	Source / Destination	Description
HCLK	IN	Clock Generator	AHB Bus Clock
HRESETn	IN	Reset Controller	Bus reset (active low).
HSELreg3	IN	Decoder	Slave select for register accesses
For each of the 4 ports ($n = 0-3$), the following signals are repeated.			
HSIZE $n[1:0]$	IN	Master	Indicates the current transfer size.
HWRITE n	IN	Master	When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral.
HTRANS $n[1:0]$	IN	Master	AHB Transfer type
HBURST $n[2:0]$	IN	Master	AHB Burst type
HREADY inn	IN	AHB Bus	AHB transfer done
HSELram n	IN	Decoder	Slave select for memory accesses
HADDR $n[28:0]$	IN	Master	This is part of the AHB address bus, which is used for decoding the internal address space.
HWDATA $n[31:0]$	IN	Master	AHB Write Data bus.
HRDATA $n[31:0]$	OUT	AHB Bus	Read Data to AHB
HRESP $n[1:0]$	OUT	Master	AHB transfer response
HREADY $outn$	OUT	Master	AHB transfer done

Table 4.2-1 AHB signal descriptions

Signal Name	Type	Source / Destination	Description
nPOR	IN	Reset Controller	Power on Reset.
SREFReq	IN	PMU	Self Refresh Request
SREFAck	OUT	PMU	Self Refresh Acknowledge
ExtBusReq	OUT	External Bus Arbiter	Bus Request for External Memory address and data bus
ExtBusGnt	IN	External Bus Arbiter	Bus Grant for External Memory address and data bus
ExtCtlReq	OUT	External Bus Arbiter	Bus Request for External Memory control bus
ExtCtlGnt	IN	External Bus Arbiter	Bus Grant for External Memory control bus
SCANENABLE	IN	Test Controller	This signal is used by the Test Controller during Scan testing to shift in and shift out Scan data.
SCANIN	IN	Test Controller	Scan Data Input

SCANOUT	OUT	Test Controller	Scan Data Output
BIGENDIAN	IN	System	Sets all AHB ports to BigEndian byte lane order

Table 4.2-2 On-Chip signals

Signal Name	Type	Source / Destination	Description
CLKIn	IN	System	Fed back version of the clock to the external SDRAMs.
CLKOut	OUT	SDRAM	Clock output to the SDRAMs.
DataIn[31:0]	IN	SDRAM	Data input from the SDRAMs
DataOut[31:0]	OUT	SDRAM	Data output to the SDRAMs
DQMOOut[3:0]	OUT	SDRAM	Data Mask output to the SDRAMs
DataEn	OUT	PAD	SDRAM pad enables.
AddrOut[14:0]	OUT	SDRAM	Address output to the SDRAMs
CKEOOut[3:0]	OUT	SDRAM	Clock Enable outputs to the SDRAMs
nCSOut[3:0]	OUT	SDRAM	Chip Selects to the SDRAMs
nRASOut	OUT	SDRAM	Row Address Strobe to the SDRAMs
nCASOut	OUT	SDRAM	Column Address Strobe to the SDRAMs
nWEOOut	OUT	SDRAM	Write Enable to the SDRAMs.

Table 4.2-3 Signals to I/O pad cells

4.3 Bus Interconnection

The SDRAM Controller block interfaces with the AMBA AHB interface using separate uni-directional read (output) and write (input) data buses HRDATA and HWDATA. For further information refer to the AHB Specification [Ref.1].

The data bus may be interfaced to the Bus Masters either using multiplexers or logical OR'ing of data buses from other AHB peripheral blocks. In common with other AHB peripheral blocks, the SDRAM Controller block forces all read data bus lines low when there are no AHB read transfers in progress.

However, it is also possible to interface the SDRAM Controller block with a 3-state bi-directional data bus. A wrapper needs to be added around the SDRAM Controller block connecting the read and write data buses to bi-directional drivers. This is best done by explicit instantiation of 3-state driver cells in the HDL code. Control of the 3-state drivers can be done using the AHB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Endianness

The SDRAM Controller AHB ports can operate in systems using either little-endian or big-endian byte lane order. Byte lane order is set by the BIGENDIAN input signal.

Internal operation of the SDRAM Controller always uses little-endian byte order. When the BIGENDIAN signal is asserted the two least significant address lines of each AHB port are inverted. This converts the byte lane endian order of AHB transfers to BigEndian. For byte reads, the SDRAM controller duplicates data on all byte lanes. Similarly half word reads duplicate data on both half words.

4.5 Customisation

The ARM PrimeCell SDRAM Controller can be modified to suit specific user configurations, but the full benefit of a fast integration will only be obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the SDRAM Controller (PL170) Design Manual [Ref.3].

4.5.1 SDRAM Timing Parameters

SDRAM Device timing parameters are defined in the SdramDefs module. The user is required to refer to the datasheet of the SDRAM memory devices being used in the system and should set these parameters to suitable values.

Table 4.5-1 lists the timing parameters.

Parameter	Description	Default value (decimal)	Allowable range
tRP	Precharge to ACT command period	3	0 to 4
tRC	Ref/ACT to Ref/ACT command period	9	0 to 16
tRAS	ACT to Precharge command period	6	0 to 8
tRRD	ACT to ACT (different bank)	2	0 to 4
tSREX	Self-Refresh exit time	2	2 to 8
tXSR	Self-Refresh exit to ACT command	9	tSREX to (tSREX+17)
tAPW	Last data in to any command to same device (with AutoPrecharge)	5	0 to 8
tAPR	Last data out to any command to same device (with AutoPrecharge)	1	1
tRWL	Last data in to Precharge	2	0 to 8
tRSA	Mode command to ACT period	3	0 to 4

Table 4.5-1 List of SDRAM timing parameters

4.5.2 Other parameters

NO_OF_WRITE_BUFFERS : This parameter specifies the number of quad-word write buffers required in the module. The allowable values for this parameter are 0 and 2. If any other value is specified, then the design will not contain any write buffers. If the user has specified that write buffers are not required, the synthesised netlist does not contain flip-flops for the buffers.

NO_OF_READ_BUFFERS : This parameter specifies the number of quad-word write buffers required in the module. The allowable values for this parameter are 0 and 1. If any other value is specified, then the design will not contain any read buffers. If the user has specified that read buffers are not required, the synthesised netlist does not contain flip-flops for the buffers.

ADDRESS_MAPPING : This parameter controls the address mapping function in the AHB interface. If this parameter is set to 1'b1 then Address mapping is enabled, else the AHB address bus HADDR[26:2] is directly driven on the memory address bus AddrIn3[26:2]. The default value is 1'b1.

4.6 Synthesis

The SDRAM Controller block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts provided. The final area and gate count that will be achieved is dependent on the target cell library used.

A README text file in the `synopsys_ref/` directory provides details of the synthesis directory structure used.

The synthesis scripts consist of the following three main categories:

4.6.1 Setup Scripts

Before synthesis, modify the `setup.scr` file located in the `synopsys_ref/scr_common` directory. Some of the parameters that need to be set include

- a) The chosen target, link and symbol libraries.
- b) The UNIX directory path for synopsys read and write caches.

The command script includes the `cb25dontuse.scr` script which specifies cells in the target library that should not be used. This should be done as per library vendor instructions or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. It is the responsibility of the user to ensure that the `cb25dontuse.scr` files are modified and renamed appropriately for the chosen technology library.

The setup script includes the `cb25v2.1settings.scr` file. The selected technology library and operating conditions may be set in this file.

The setup script includes the `name_rules.scr` file. This file contains naming rule sets which are applied at the end of the synthesis process – prior to netlist write out. Changing names of internal nets and cells may be necessary to make the sdf files compatible with the netlist for the purpose of gate level simulations.

4.6.2 Command Script

Synthesis is performed by invoking the `syn_sdram` script from the `scr_sdram` directory. The `syn_sdram` shell script contains the following options that a user may modify:

`dc_command` Unix variable.

This variable needs to be modified by the user to point to the path of the '`dc_shell`' utility on the user's system.

`SDFILES` Unix variable

This variable contains a list of the files that need to be compiled to create the final netlist. It is recommended that the file list and the order is not modified by the user unless he/she wants to experiment with the compilation of single modules.

`HDL_IN` / `HDL_OUT` variables

These variables control the language in which the input source and the final netlist is written out.

The `syn_sdram` script invokes the master compile script `sdram.cmd`. This script analyses the HDL source before mapping and optimising to a specific target technology cell library. The script `sdram.cmd`, includes other scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys `.db` format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate `log/` directory.

Prior to a synthesis run, the directory path needs to be set for the `design_root` variable in the command script. The synthesis scripts declare file locations and search paths which are relative to the `design_root` variable. There are several methods that can be used:

- a) Use of an absolute directory path
`design_root = /peripherals/sdram_pl170`
 Although simple to use, scripts are less portable requiring modification if directory paths change.
- b) Use of a relative directory path
`design_root = .`
 Again, simple to use, but scripts must be executed from the directory which contains the master compile script.
- c) Use of an environment variable \$PERIPH
`design_root = get_unix_variable("PERIPH")`
 This requires use of the UNIX `setenv` command to set the variable PERIPH to the desired directory. It is also the same method that has been used within UNIX shell scripts that run simulations.

4.6.3 Constraint Scripts

- a) Operating conditions and signal load/drive values should be set for the chosen technology library in the `global.scr` file.
- b) Timing constraints for AHB bus signals are set via the `ahb_params.scr` and the `ahb_slave.scr` file. As delivered, the timings are set for a 90 MHz clock. These timings should be adjusted according to the target operating frequency expected for the ASIC cell library. The `ahb_params.scr` file has been written using parameters and only requires amendment of the clock frequency, all other bus parameters (except clock skew) scale accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values could be applied:

- set the minus uncertainty value to be 5% of the clock period.
- set the plus uncertainty value to be 40% of the minus uncertainty value.

The timing parameters assume a bus timing budget of 60% for output delays and 60% for input setup.

- c) Timing constraints for non-ARM PrimeCell SDRAM Controller-specific signals are set via the `sdram.scr` file. Sample input and output delay constraints are also specified for the other signals. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

4.7 Integration with AMBA Micropack components

The SDRAM Controller block is designed to be integrated into an AMBA system via the AHB (Advanced High Performance Bus). The controller can be programmed by a CPU via Port 3. The remaining 3 Ports also implement AHB slave interfaces but are considered as DMA ports since they do not provide access to the programming interface of the SDRAM Controller. External DMA Controllers implementing the AHB Master protocol may access the memory via these three ports.

4.8 Clocking

HCLK is the main clock on which the SDRAM Controller operates. This clock is also driven as the CLKOut signal to the SDRAMs. The generation of the CLKOut signal includes logic to shutdown the clock when the SDRAMs are not being accessed.

CLKIn is a fed-back version of the CLKOut signal generated by the controller. All SDRAM signals driven off-chip will be synchronized to the rising edge of CLKIn to compensate for pad and track delays. Similarly all SDRAM input signals will be synchronized to the rising edge of CLKIn before being used in the HCLK domain.

CLKIn must be controlled in chip/board layout such that it has a MIN delay of about 2-3 ns from HCLK.

4.9 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default SDRAM Controller synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.10 Resets

The SDRAM Controller block has two reset inputs. The system bus reset signal HRESETn, resets the AHB interface. A separate reset pin nPOR (Power-On-Reset), is used to reset the core logic in the SDRAM Controller. Use of a separate Power-On-Reset signal enable the contents of the memory system to be preserved over a soft reset. The resets can be asserted asynchronously. For systems which have a single reset, HRESETn can be used to drive nPOR.

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

4.11 Test Methodologies

The SDRAM Controller is designed to be tested using Scan insertion and ATPG.

4.11.1 Scan Insertion and ATPG

It is assumed that users will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The SDRAM Controller design fully supports this scan test and ATPG methodology. Only the rising edges of clocks are used in the design.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or by the use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create a netlist with scan test D flip-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven since gate-level timing will now be different.

The main advantage of the scan Insertion and ATPG approach is in cases where significant modifications have been made to the SDRAM Controller block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that a very high fault coverage can be achieved (99%).

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running automatically generated vectors (ATPG).

APPENDIX A ADDRESS MAPPING

A.1 Address Mapping turned ON

The mapping of the on-chip address to the off-chip address should typically satisfy the following two constraints:

- Each target external memory device should appear contiguous in memory.
- The bank address should be changed on the page address.

A.1.1 Address mapping

Table A.1-1 shows the mapping between the host bus address of the DMA block and the AddrInx[28:2] address lines of the SDRAM Controller, to which the DMA block connects. Note that the AddrInx lines are *internal* to the SDRAM controller top level and correspond to the port addresses as seen by the Engine. This address is interpreted by the Control Engine as below :

AddrInx[28:2] : Address input from Port X to the SDRAM control engine

AddrInx[28:27] : Device Address

AddrInx[26] :

- **Higher** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **256 Mbit** devices
- **Lower** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **non-256 Mbit** but **4-bank** devices
- 0 (zero) if device on slot pointed by AddrInx[28:27] is populated with **2-bank** devices.

AddrInx[25] :

- **Lower** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **256 Mbit** devices
- **Higher** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **non-256 Mbit** but **4-bank** devices
- Bank address bit if device on slot pointed by AddrInx[28:27] is populated with **2-bank** devices.

AddrInx[24] :

- Row address MSB if device on slot pointed by AddrInx[28:27] is populated with **256 Mbit** devices
- **Lower** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **non-256 Mbit** but **4-bank** devices
- 0 (zero) for 2-bank devices

AddrInx[23] :

- Row Address (MSB – 1) if device on slot pointed by AddrInx[28:27] is populated with **256 Mbit** devices
- Row address MSB if device on slot pointed by AddrInx[28:27] is populated with **non-256 Mbit** but **4-bank** devices
- Bank Address bit for 2-bank devices

AddrInx[22:12] : Row Address

AddrInx[11:2] : Column Address

Ext Bus Wld	Device		Row/ Column	Device Address		Bank Address		RowAddress														ColumnAddress														Mem cap
	Cap.	Data bus		A 28	A 27	A 26	A 25	A 24	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2						
32	64 M bit	x32	Row2048 Bank4 Ccl256	A 28	A 27	A 11	A 10	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 25	A 24	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	32M Byte					
			x16	Row4096 Bank4 Ccl256	A 28	A 27	A 11	A 10	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 25	A 24	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	64M Byte				
			x8	Row4096 Bank4 Ccl512	A 28	A 27	A 11	A 12	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 25	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	128M Byte				
	128 M bit	x16	Row4096 Bank4 Ccl512	A 28	A 27	A 11	A 10	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 25	A 24	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	128M Byte					
			x8	Row4096 Bank4 Ccl1024	A 28	A 27	A 11	A 12	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 25	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	256M Byte				
	256 M bit	x16	Row8192 Bank4 Ccl512	A 28	A 27	A 11	A 12	A 24	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 26	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	256M Byte					
			x8	Row8192 Bank4 Ccl1024	A 28	A 27	A 11	A 12	A 24	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 26	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	512M Byte				
		16 M bit	x16	Row2048 Bank2 Ccl256	A 28	A 27	0	A 10	0	A 10	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 25	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	16M Byte					
x8				Row2048 Bank2 Ccl512	A 28	A 27	0	A 11	0	A 11	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 25	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	32M Byte				

Table A.1-1 Mapping between System Bus Address A[31:0] and AddrInx[28:2] (Shaded regions indicate address replication specifically done to minimize muxing)

A.2 Address Mapping turned OFF

When Address Mapping is turned OFF, the address on the Bus is presented directly onto AddrInx[28:2]. This address is interpreted as shown in Section A.1. When Address Mapping is turned OFF, the DMA interface needs to implement the following routing of the Bank address bits :

For accesses targeting 16 Mbit devices (2-bank devices, B bit in Configuration register 0 is cleared), the DMA interface needs to implement the following mapping:

- AddrInx[26] and AddrInx[24] to be driven to 0
- Bank address on AddrInx[25] to be replicated on AddrInx[23]

This mapping is required because 16Mbit (2-bank) devices require the Bank Address to be driven on the A11 device address line. Since AddrInx[23] is driven on A11 when address mapping is turned OFF, it is required that the DMA interface drive the Bank Address on AddrInx[25] and AddrInx[23].

The SDRAM controller uses AddrInx[26:25] for bank match-mismatch calculations irrespective of the device type (2-bank/4-bank). Hence, for 2-bank devices, it is required that AddrInx[26] and A[24] be driven to 0.

For accesses targeting non-256Mbit, but 4-bank devices (B = 0 and F = 0), the DMA interface needs to implement the following mapping:

- Bank address on AddrInx[26] to be replicated on AddrInx[24]

For accesses targeting 256 Mbit devices, no such mapping is required.

Note that these mappings are applicable only when Address Mapping is turned OFF. When Address Mapping is turned ON, the mapping shown in Table A.1-1 applies.